Sistemas Informáticos II

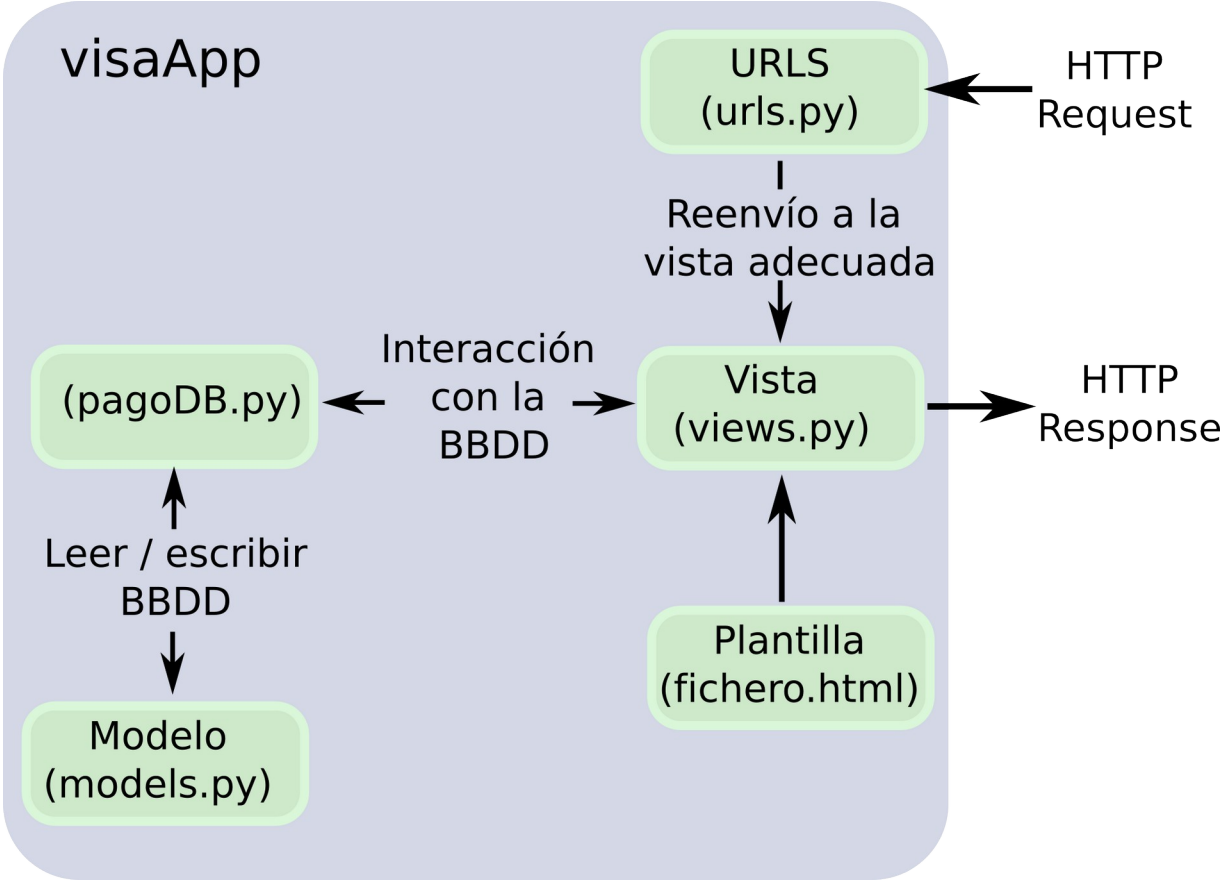
P2

Objetivos

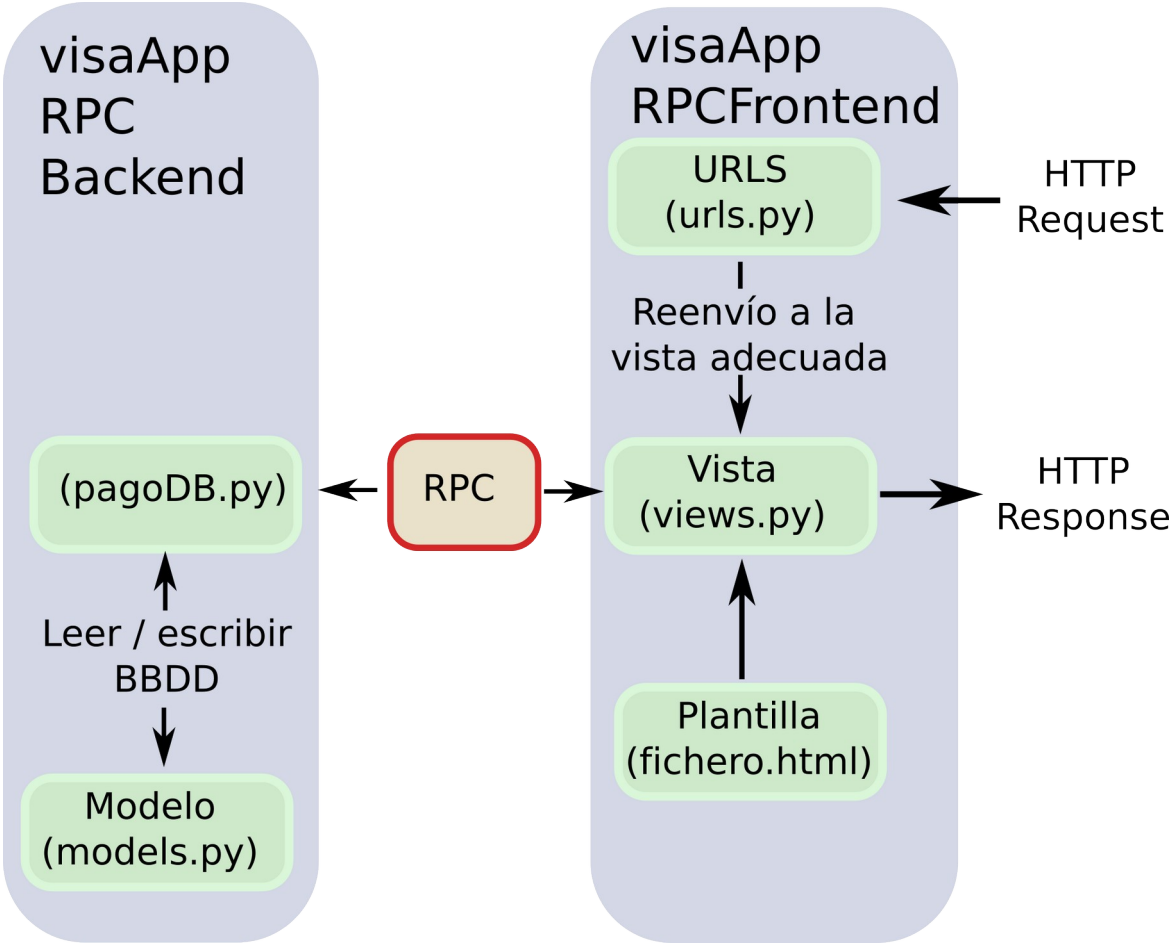
Familiarizarse e implementar mecanismos de comunicación basados en:

- **RPC**: permite invocar procedimientos remotos como si fueran locales, con transparencia de datos.
- **Colas de Mensajes**: que introducen una capa de asincronía y desacoplamiento entre los componentes.

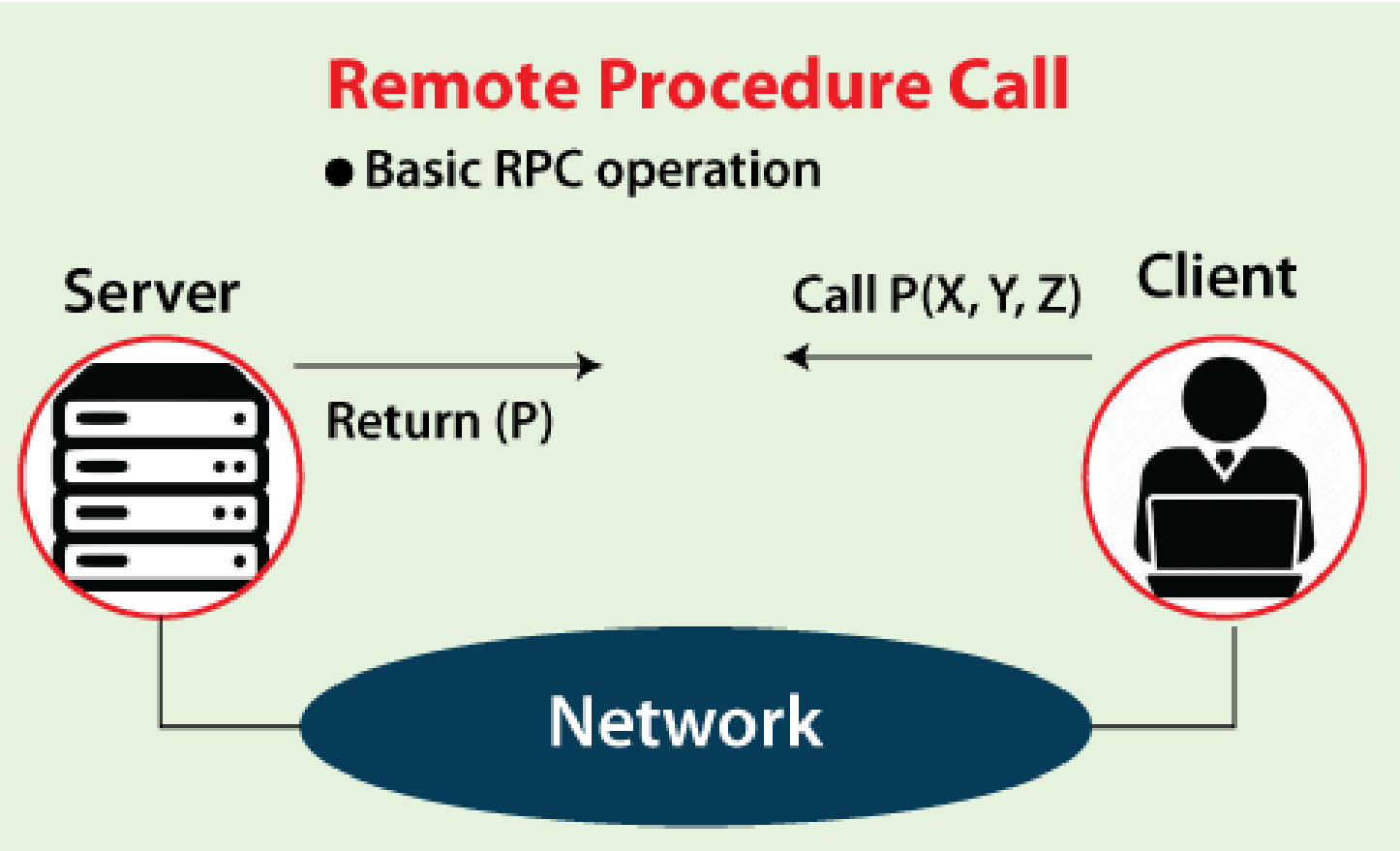
Aplicación de Partida



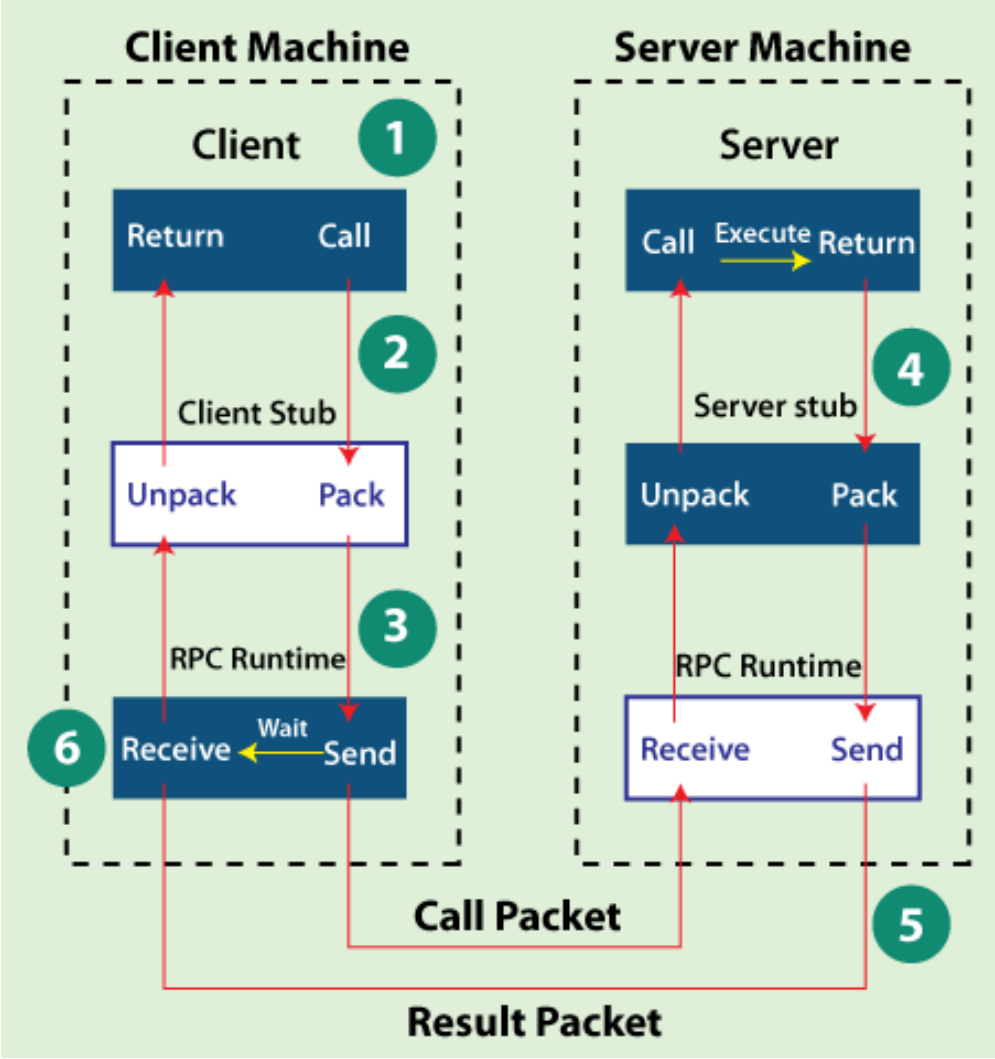
Aplicación Separada en Dos



Funcionamiento RPC



Componentes RPC



Modern RPC

Biblioteca para Django que permite implementar de manera sencilla y eficiente servicios RPC:

- Abstrae las complejidades de la comunicación subyacente, con características avanzadas como uso de decoradores (@rpc_method).
- Soporta dos protocolos de comunicación: JSON-RPC y XML-RPC.

Nosotros nos vamos a centrar en XML-RPC

XML-RPC

Protocolo de comunicación simple que permite la interacción remota entre aplicaciones mediante la transmisión de datos en XML.

- Usa HTTP para el transporte de mensajes.
- Formato legible y estandarizado para el intercambio de información.
- Proporciona transparencia de datos.
- Puede ser menos eficiente en comparación con alternativas modernas JSON-RPC debido a la sobrecarga del formato XML.

XML-RPC - Petición

```
<? xml version ="1.0"? >
< methodCall >
< methodName > org . wikipedia . intercambioDatos
</ methodName >
< params >
< param >< value > < i4 >360 </ i4 > </ value >
</ param >
< param >< value > < i4 >221 </ i4 > </ value >
</ param >
</ params >
</ methodCall >
```

XML-RPC - Respuesta

<? xml version ="1.0"? >
< methodResponse >
< params >
< param >
< value > < string > Intercambio datos nro . 360 por
221 </ string > </ value >
</ param >
</ params >
</ methodResponse >

Modern RPC y XML-RPC

Permiten llevar a cabo la lógica de la aplicación de forma remota:

- Comprobación de la información de la tarjeta.
- Registro del pago.
- Borrado de pagos.
- Listado de pagos.

Colas de Mensajes

- Usan dispositivos intermedios de almacenamiento.
- Permiten comunicación asíncrona.
- Filtrado, prioridades, etc.

RabbitMQ

Plataforma de mensajería robusta y versátil que implementa el protocolo de mensajería AMQP (Advanced Message Queuing Protocol)

- Compatible con múltiples lenguajes de programación (Python).
- Características avanzadas como la persistencia de mensajes, gestión de prioridades, tolerancia a fallos mediante clústeres y réplicas, y extensibilidad a través de plugins.

Pika

Paquete de Python ligero y flexible diseñado para interactuar con sistemas de mensajería basados en el protocolo AMQP, como RabbitMQ.

- Permite crear y gestionar conexiones, canales y colas.
- Permite desarrollar patrones de mensajería como publicación/suscripción, enrutamiento o mensajes punto a punto de forma sencilla y eficiente.

Servicio Cancelación de Pago

Permite cancelar un pago.

- El servidor atiende los mensajes depositados en una cola.
- Cada mensaje contiene el ID del pago a cancelar.
- Una vez recibido un mensaje el servidor debe actualizar su código de respuesta a “111”.
- El cliente únicamente coloca mensajes en la cola con el ID del pago a cancelar.

Entrega

- Dos semanas de tiempo para realizarla (revisar calendario de prácticas).
- A modo de resumen, para esta práctica se espera encontrar dentro del archivo SI2P2_<grupo>_pareja>.zip.
  - Memoria de la práctica.
  - Carpetas P1-rpc-backend y P1-rpc-frontend.
  - Fichero del cliente y servidor del servicio de cancelación de pago.